Performance Testing

Date	03 July 2026
Team ID	
Team Size	3
Team Leader	Chekali Basavaraju
Team Member 1	Girish C
Team Member 2	Shaik Sameer
Team Member 3	
Team Member 4	
Project Name	Smart Lender: Machine Learning-Based Loan Approval Prediction System
Maximum Marks	

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	JMeter
Type of Testing	Load Testing, Stress Testing, Spike Testing
Target Module I API	Credit Card Approval Prediction API (/predict)
Test Environment	Localhost (Python Flask Server on Local Machine)
Test Date	03 July 2026

Step 2: Test Scenarios

S.No	Test Scenario I Description	No. of Virtual Users	Duration (sec)	Expected Outcome
	Normal Load Test — Multiple users accessing prediction API	10	60	System should respond within acceptable time with 100% success
2	High Load Test — Simulate heavy concurrent users	50	120	System should handle load with response time < 2 sec and high success rate
3	Stress Test — Push system beyond normal capacity	100	180	System should remain stable and not crash; graceful handling of requests
4	Spike Test — Sudden increase in user load	200	60	System should handle spikes with minimal performance degradation

Step 3: Performance Test Results

s.No	Metric	Target Value	Actual Value	Status (Pass I Fail)	Remarks
	Response Time (Avg)	< 2 seconds	1.18 seconds	Pass	Average response time is within acceptable limit.
2	Response Time (Max)	< 5 seconds	2.86 seconds	Pass	Maximum response time is within acceptable limit.
3	Throughput (Req/sec)		148.7 req/sec	Pass	System handled high request rate successfully.
4	Error Rate		0.32%	Pass	Error rate is less than 1%.

5	CPU Utilization		63%	Pass	CPU usage is under 80% during test.
6	Memory Utilization		56%	Pass	Memory usage is under 80% during test.

Step 4: Observations & Analysis

Key Findings

- The system maintained response times within acceptable limits for both average and maximum thresholds.
- Throughput remained stable, indicating efficient handling of concurrent user requests.
- Error rate stayed below 1%, demonstrating reliable and consistent system performance.

Bottlenecks Identified

- Increased database query time during peak concurrent user requests.
- Minor delay observed in dashboard analytics due to intensive data processing.

Optimization Steps Taken

- Optimized SQL queries and added indexing to frequently accessed tables.
- Implemented caching mechanisms for dashboard data to improve response time.
- Enabled database connection pooling to enhance performance under concurrent load.

Step 5: Screenshots I Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

[Insert Screenshot 1 here — JMeter Summary Report]

[Insert Screenshot 2 here — Response Time Graph]